

INF-2700: Database systems

Assignment 2

Jens Christian Valen Leynse

jeley4465

jeley4465@post.uit.no

Oct 16th, 2023

Introduction

The assignment given is to implement some basic elements of database management systems (DBMS) in c. How they work, how to measure them with analysis tools and finally understanding the database performance. The assignment is split into 4 tasks. Task 1 retains to understanding the base program, so there isn't any output. Task 2 extending the types of queries the program can handle. Task 3 is implementing binary search as the code uses linear. Task 4 and the final task is comparing the solution made in task 3 with a B⁺-tree organized file.

Requirements:

- Query must be able to handle all comparison searches, namely =, <, <=, >, >= and !=.
- Generating a table of an arbitrary size.
- Performance data for the two search algorithms.

Design & Implementation

Task 2: Extending the types of queries

There is already a function that handles equal operations on integer attributes for the inputted queries. To include additional operator functionality, such as <, <=, >, >=, and !=, we can extend the function by incorporating a simple comparison loop. This loop will manage all operators by calling their corresponding logic functions.

Task 3: Binary search

The objective of this task is to compare a linear search algorithm with a binary one. To achieve this, the `table_search` function was modified to accept an additional argument – the search type, which simplifies the distinction between the two methods.

To provide a step-by-step explanation of the process, the `testmain` file was expanded to include functions for creating and populating test tables, as well as executing and analyzing the results. Both search algorithms are then tested with the same data.

Inside the `table_search` function, a schema is created through either dynamic allocation or a fixed-sized array (both methods are tested below). Depending on the search type argument provided during the function call, the corresponding search method is executed.

The binary search function operates by iteratively identifying the middle of the list and determining whether the value being sought is larger or smaller in comparison. It then recursively calls itself with the list halved to further track the target number.

Task 4: Comparison with B⁺-tree

The B⁺-tree stores all data in its leaf nodes, creating a balanced tree. This design reduces the tree's height as it can accommodate more nodes.

In terms of storage, the B⁺-tree includes pointers between leaf nodes to fill any gaps. These pointers effectively establish a linked list at the leaf level. Therefore, the query doesn't need to be sorted in a specific order, as the pointers lead to the next value in the sequence. However, the index information (without the actual values) must also be stored. This means that the sorted queue in the B⁺-tree must be larger than the binary queue.

In the case of binary search, it already knows its size, as it operates on a single queue where all data is accessible at the same level.

Since data is only found in the leaf nodes of the B⁺-tree, it needs to iterate through the indexes first to locate the values. In contrast, binary search compares values at the same level, moving back and forth.

In terms of sorting notation, the B⁺-tree is represented as " $\log n/2$ " with memory usage of " n ," while binary search has the notation " $\log_2 n$ " with the same memory usage of " n ." Here, " n " represents the number of records. As a result, for a database, B⁺-trees provide a more efficient lookup, as $n/2 > 2$.

Results for the profiling

To execute the code, follow the instructions from the README file.

Dynamic allocation

Number of disk seeks/ reads/ writes/ IOs on 100:				
	first	half	missing	Random ($^{2/3}$)
Binary search	1/1/0/1	1/2/0/2	1/2/0/2	1/2/0/2
Linear search	1/2/0/2	1/2/0/2	1/2/0/2	1/2/0/2

Number of disk seeks/ reads/ writes/ IOs on 10000:				
	first	half	missing	random($^{2/3}$)
Binary search	7/8/0/8	7/9/0/9	7/9/0/9	8/9/0/9
Linear search	1/164/0/164	1/164/0/164	1/164/0/164	1/164/0/164

Fixed-sizes array

Number of disk seeks/ reads/ writes/ IOs on 100:				
	first	half	missing	Random ($^{2/3}$)
Binary search	1/1/0/1	1/2/0/2	1/2/0/2	1/2/0/2
Linear search	3/3/0/3	3/3/0/3	1/2/0/2	2/3/0/3

Number of disk seeks/ reads/ writes/ IOs on 10000:				
	first	half	missing	random($^{2/3}$)
Binary search	7/8/0/8	7/9/0/9	7/9/0/9	8/9/0/9
Linear search	3/165/0/165	3/165/0/165	1/164/0/164	3/165/0/165

In terms of disk operations, all tests exhibit similar patterns, with reads and IOs having the same number of operations, and seek operations occurring with fewer occurrences, indicating the number of times the index is moved to find the target data. Notably, fixed-size arrays results in a significantly higher number of seek operations and there's no write output since the output is generated by a search algorithm that doesn't modify anything.

Consistently, dynamic allocation requires fewer disk operations compared to fixed-size arrays. This suggests that dynamic allocation is more efficient, as it avoids wasting memory by allocating a fixed and potentially excessive amount of space.

In scenarios where we search for elements at the "first" or "half" positions, the least number of disk operations is needed due to these elements' easily accessible positions. The "missing" scenario, which involves searching for an element outside the data range, has the fewest disk operations. The "random ($2/3$)" scenario, involves searching for an element at a random position in the data, the chosen position was intended to challenge binary search , as it's the last available index after halving the list multiple times.

As expected, searching for elements in a larger dataset (10,000 elements) requires more disk operations than in a smaller dataset (100 elements) for both dynamic allocation and fixed-size arrays. However, binary search consistently requires fewer disk operations than linear search in all scenarios and for both dynamic allocation and fixed-size arrays, thanks to its more efficient search strategy.

Discussion

A couple of minor issues were encountered during code writing and testing:

Regardless of the file type used (txt, sql, dbcmd), the SQLite shell had difficulties processing queries in the same way as manual inputs. For larger tables, the shell would sometimes

issue warnings about reaching the limit of open files, or it would produce an error indicating that it failed to obtain input. Despite these issues, it was possible to overlook them since the output was accurate until it reached the point where it struggled with the data.

An attempt was made to implement a higher-level binary search algorithm. However, this approach yielded less usable output, and more often than not, it resulted in a segmentation fault.

Conclusion

In conclusion, the data suggests that dynamic allocation is generally more efficient than using fixed-size arrays, and binary search is more efficient than linear search in terms of disk operations. The efficiency of these operations is influenced by the size of the dataset and the specific search scenario. In general, the database management systems have been better understood throughout the process and the requirements for each task have been implemented in a working manner as the program can run while performing the various tasks.